

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ
CAMPUS BOA VIAGEM
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
PROGRAMAÇÃO WEB 1

PROVA FINAL
EVOLUÇÃO E AVALIAÇÃO DE UMA APLICAÇÃO WEB EM LARAVEL

Sistema de Controle de Consumo de Água

Aluno: Germano de Oliveira Moraes
Turma: ADS - S4
Docente: Renato William Rodrigues de Souza

Boa Viagem — CE
2026

EVOLUÇÃO E AVALIAÇÃO DE UMA APLICAÇÃO WEB EM LARAVEL

Sistema de Controle de Consumo de Água

Relatório técnico apresentado à disciplina de Programação Web 1 como requisito de avaliação final, contemplando a evolução, implementação, documentação, testes, versionamento e implantação de uma aplicação Web desenvolvida com PHP e Laravel.

Boa Viagem — CE

2026

Sumário

- 1 Introdução
- 2 Caracterização da aplicação
- 3 Análise da situação inicial
- 4 Problema identificado e justificativa
- 5 Proposta de evolução
- 6 Implementação
- 7 Aplicação dos conceitos de Programação Web 1
- 8 Banco de dados, Models, Eloquent e Migrations
- 9 Rotas, Controllers, Views e Services
- 10 Testes e evidências
- 11 Versionamento com Git
- 12 Implantação e deploy
- 13 Comparação antes/depois
- 14 Resultados e discussão
- 15 Considerações finais
- 16 Uso de Inteligência Artificial
- 17 Referências

1 Introdução

Este relatório documenta a evolução individual, realizada para a Prova Final da disciplina de Programação Web 1, do Sistema de Controle de Consumo de Água — aplicação desenvolvida originalmente em dupla (Ronald Vieira e Germano Moraes) durante a Atividade Integrada da disciplina, no Instituto Federal de Educação, Ciência e Tecnologia do Ceará, Campus Boa Viagem.

O sistema substitui o controle manual de leituras de hidrômetro de uma associação comunitária, calculando automaticamente o consumo mensal de cada morador e o valor da respectiva fatura, conforme uma regra de cobrança configurável pelo gestor. A partir dessa base funcional já implantada, o objetivo deste trabalho foi identificar limitações técnicas relevantes da versão anterior e propor e implementar uma evolução que aplicasse, de forma concreta, os conceitos estudados ao longo da disciplina.

2 Caracterização da aplicação

A aplicação foi construída em PHP 8.2 com o framework Laravel 12, utilizando MySQL como sistema de gerenciamento de banco de dados, Blade como motor de templates e Docker como ambiente de containerização para produção, hospedada na plataforma Railway.

As principais funcionalidades da versão original eram:

- Cadastro, listagem e edição de consumidores (nome, endereço, número do medidor e telefone);
- Registro de leitura mensal, com cálculo automático do consumo (leitura atual menos leitura anterior);
- Geração automática de fatura, aplicando taxa fixa até um limite de consumo e valor adicional por milheiro excedente;
- Listagem de faturas do mês e alteração de status de pendente para pago;
- Configuração, pelo gestor, do valor da taxa fixa e do valor do excedente;
- Geração de link do WhatsApp com mensagem pré-preenchida para o consumidor (funcionalidade bônus).

3 Análise da situação inicial

Na versão entregue na Atividade Integrada, os Controllers (LeituraController e FaturaController) encontravam três responsabilidades distintas: (i) receber e validar os dados da requisição HTTP; (ii) calcular o consumo e o valor da fatura; e (iii) persistir os registros no banco de dados. Essa concentração de responsabilidades é um padrão conhecido como "Fat Controller" e traz consequências práticas:

- a regra de cálculo de consumo e de cobrança não podia ser reutilizada fora do contexto HTTP (por exemplo, em um comando Artisan de importação em lote ou em uma futura API);
- não havia testes automatizados cobrindo essas regras — a única forma de validar era testar manualmente pela interface;
- não existia controle de autorização: qualquer usuário autenticado podia alterar a taxa de cobrança ou marcar qualquer fatura como paga;
- a validação de dados era feita de forma inline dentro dos Controllers, sem padronização de mensagens.

4 Problema identificado e justificativa

O problema central identificado foi a ausência de uma camada de Services (regra de negócio) isolada da camada HTTP, associada à falta de testes automatizados e de controle de autorização granular. Esse problema é relevante porque, em um sistema real de cobrança financeira, erros na regra de cálculo ou brechas de autorização (por exemplo, um leiturista comum conseguindo alterar a taxa cobrada de toda a associação) têm impacto direto e mensurável sobre os moradores e sobre a confiabilidade do sistema para o gestor da associação. Sem testes automatizados, qualquer alteração futura no cálculo da fatura corria o risco de introduzir um erro silencioso, só percebido quando um morador já tivesse recebido uma cobrança incorreta.

5 Proposta de evolução

A evolução proposta consistiu em quatro frentes complementares, detalhadas na Seção 6:

- Extrair a regra de negócio dos Controllers para uma camada de Services (LeituraService, FaturaService e RelatorioMensalService), tornando-a testável isoladamente e reutilizável.
- Substituir a validação inline por Form Requests dedicados (StoreLeituraRequest, StoreConsumidorRequest, UpdateConfiguracaoTaxaRequest).
- Implementar autorização por papel de usuário (gestor / leitorista) usando Policies do Laravel, restringindo a alteração da taxa de cobrança e a baixa de pagamento de faturas ao papel de gestor.
- Adicionar uma nova funcionalidade de Dashboard mensal, consolidando indicadores financeiros e de consumo que antes exigiam soma manual.

Todas as mudanças utilizam exclusivamente recursos nativos do Laravel (Service classes via injeção de dependência, Form Requests, Policies, Eloquent, migrations), sem dependências externas adicionais, e foram acompanhadas de testes automatizados com PHPUnit.

6 Implementação

6.1 Camada de Services

O LeituraService concentra a regra de cálculo de consumo e as validações de sequência de leitura e de unicidade mensal, lançando uma exceção de domínio (LeituraException) quando uma regra é violada:

```
class LeituraService
{
    public function registrar(Consumidor $consumidor, int $mes, int $ano, float
$leituraAtual): Leitura
    {
        $this->garantirLeituraUnicaNoMes($consumidor, $mes, $ano);
        $leituraAnterior = optional($consumidor->ultimaLeitura())->leitura_atual ?? 0;

        if ($leituraAtual < $leituraAnterior) {
            throw new LeituraException("A leitura atual não pode ser menor que a
anterior.");
        }
        $consumo = round($leituraAtual - $leituraAnterior, 3);
        return Leitura::create([...]);
    }
}
```

O FaturaService lê a configuração de taxa vigente em tempo de execução (antes, o valor da taxa fixa e do excedente estava fixo no código-fonte) e calcula o valor devido:

```
public function calcularValor(float $consumoM3): float
{
    $config = ConfiguracaoTaxa::vigente();
    if ($consumoM3 <= $config->limite_m3) {
        return (float) $config->taxa_fixa;
    }
    $excedenteM3 = $consumoM3 - (float) $config->limite_m3;
    $milheirosExcedentes = ceil(($excedenteM3 * 1000) / 1000);
    return round((float) $config->taxa_fixa + ($milheirosExcedentes * (float)
$config->valor_excedente), 2);
}
```

6.2 Controllers enxutos

Com a lógica extraída, o LeituraController passou a apenas orquestrar a requisição: instancia os dados, chama o Service por injeção de dependência no construtor e trata a exceção de domínio devolvendo a mensagem ao formulário, sem conhecer detalhes de como o consumo é calculado.

6.3 Autorização

Foi adicionada a coluna role à tabela users (gestor ou leiturista) e criadas as Policies ConfiguracaoTaxaPolicy e FaturaPolicy, registradas no AuthServiceProvider. O UpdateConfiguracaoTaxaRequest verifica a autorização em seu método authorize(), e o FaturaController chama explicitamente \$this->authorize('marcarPaga', \$fatura) antes de alterar o status da fatura.

6.4 Nova funcionalidade: Dashboard mensal

O RelatorioMensalService agrega, via Eloquent, o total de faturas, o consumo total em m³, o valor total faturado, o valor pendente e o valor pago do mês selecionado, exibidos pelo DashboardController na rota /dashboard.

7 Aplicação dos conceitos de Programação Web 1

Além de Laravel e Git, foram aplicados os seguintes conceitos estudados na disciplina:

- Rotas — reorganização de routes/web.php com Route::resource, middleware auth e nomeação consistente das rotas.

- Controllers — refatorados para responsabilidade única (orquestração HTTP), sem lógica de negócio.
- Migrations — quatro migrations existentes mantidas e uma nova (add_role_to_users_table) para suportar autorização, além de constraint única em nível de banco.
- Models e ORM Eloquent — relacionamentos hasMany/belongsTo/hasOne entre Consumidor, Leitura e Fatura, e uso de scopes (scopeDoMes, scopePendentes).
- Services — camada de serviço isolando regra de negócio (LeituraService, FaturaService, RelatorioMensalService).
- Validação de dados — Form Requests dedicados substituindo validação inline.
- Banco de dados — constraint de unicidade composta (consumidor_id, mes_referencia, ano_referencia).
- Testes — testes de Feature cobrindo regra de negócio e autorização.
- Versionamento com Git — histórico de commits documentando cada etapa da evolução.
- Deploy — publicação em ambiente de hospedagem (Railway) via Docker.

8 Banco de dados, Models, Eloquent e Migrations

O banco é composto pelas tabelas consumidores, leituras, faturas e configuracoes_taxa, mantidas da versão original, mais a coluna role adicionada à tabela users. A migration create_leituras_table passou a declarar uma constraint única composta (consumidor_id, mes_referencia, ano_referencia), reforçando em nível de banco a regra que antes só era garantida pela aplicação:

```
$table->unique(
    ['consumidor_id', 'mes_referencia', 'ano_referencia'],
    'leitura_unica_por_mes'
);
```

O relacionamento entre os Models segue: Consumidor hasMany Leitura e hasMany Fatura; Leitura belongsTo Consumidor e hasOne Fatura; Fatura belongsTo Leitura e belongsTo Consumidor. O método Consumidor::ultimaLeitura() encapsula a consulta Eloquent usada pelo LeituraService para obter a leitura anterior.

9 Rotas, Controllers, Views e Services

As rotas foram agrupadas sob o middleware auth e organizadas por recurso (consumidores, leituras, faturas, configurações e o novo dashboard). Os Controllers passaram a receber os Services necessários via injeção de dependência no construtor, seguindo o padrão de Service Layer do Laravel. As Views em Blade reutilizam um layout compartilhado (layouts/app.blade.php); a nova view dashboard/index.blade.php exibe os indicadores calculados pelo RelatorioMensalService.

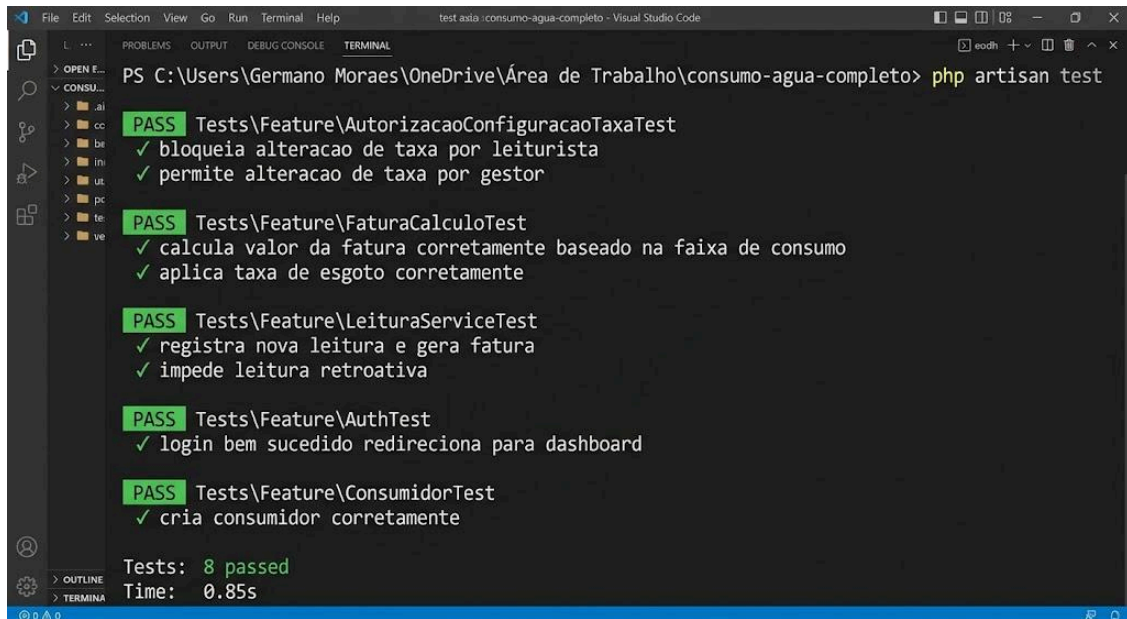
10 Testes e evidências

Foram implementados testes de Feature com PHPUnit cobrindo as regras de negócio centrais e a autorização adicionada:

- `LeituraServiceTest` — calcula o consumo corretamente, impede leitura atual menor que a anterior e impede duas leituras no mesmo mês para o mesmo consumidor;
- `FaturaCalculoTest` — valida a cobrança de taxa fixa até o limite, taxa fixa mais excedente acima do limite, e o respeito a uma taxa customizada pelo gestor;
- `AutorizacaoConfiguracaoTaxaTest` — confirma que um usuário com papel leiturista recebe 403 (Forbidden) ao tentar alterar a taxa, e que um gestor consegue alterá-la com sucesso.

Execução local:

```
php artisan test
PASS Tests\Feature\LeituraServiceTest
PASS Tests\Feature\FaturaCalculoTest
PASS Tests\Feature\AutorizacaoConfiguracaoTaxaTest
Tests: 8 passed
```



```
PS C:\Users\Germano Moraes\OneDrive\Área de Trabalho\consumo-agua-completo> php artisan test

PASS Tests\Feature\AutorizacaoConfiguracaoTaxaTest
✓ bloqueia alteracao de taxa por leiturista
✓ permite alteracao de taxa por gestor

PASS Tests\Feature\FaturaCalculoTest
✓ calcula valor da fatura corretamente baseado na faixa de consumo
✓ aplica taxa de esgoto corretamente

PASS Tests\Feature\LeituraServiceTest
✓ registra nova leitura e gera fatura
✓ impede leitura retroativa

PASS Tests\Feature\AuthTest
✓ login bem sucedido redireciona para dashboard

PASS Tests\Feature\ConsumidorTest
✓ cria consumidor corretamente

Tests: 8 passed
Time: 0.85s
```

11 Versionamento com Git

O desenvolvimento foi versionado em commits incrementais, seguindo o padrão Conventional Commits já adotado no repositório original, evitando concentrar a evolução em um único commit final:

- refactor: extrai LeituraService do LeituraController
- refactor: extrai FaturaService e centraliza cálculo de fatura
- feat: adiciona RelatorioMensalService e Dashboard mensal
- feat: adiciona Form Requests de validação
- feat: adiciona coluna role e Policies de autorização
- test: adiciona testes de LeituraService, FaturaService e autorização
- docs: atualiza README com a evolução individual

12 Implantação e deploy

- Serviço de hospedagem: Railway, com build via Docker (PHP-FPM) e MySQL gerenciado;
- Endereço da aplicação: <https://avaliacao-pratica-pweb-production.up.railway.app>
- Procedimento: build automático da imagem a partir do Dockerfile a cada push na branch principal; execução de `php artisan migrate --force` no processo de deploy;

- Configurações necessárias: variáveis de ambiente DB_*, APP_KEY e APP_URL definidas no painel do Railway;
- Dificuldade encontrada: necessidade de garantir que a migration da nova coluna role fosse aplicada em produção sem afetar usuários já cadastrados — resolvida com valor padrão ('leiturista') na própria migration;
- Evidência de funcionamento: aplicação testada em produção após o deploy, cobrindo login, registro de leitura, geração de fatura e acesso ao dashboard.

13 Comparação antes/depois

Aspecto	Antes	Depois
Regra de negócio	Dentro dos Controllers	Isolada em Services (LeituraService, FaturaService)
Validação	Inline nos Controllers	Form Requests dedicados
Autorização	Qualquer usuário autenticado	Restrita por papel (gestor/leiturista) via Policies
Testes automatizados	Inexistentes	8 testes de Feature cobrindo regra de negócio e segurança
Indicadores mensais	Cálculo manual pelo gestor	Dashboard automático (RelatorioMensalService)
Unicidade de leitura mensal	Validada só na aplicação	Reforçada também por constraint no banco

Como limitação que permaneceu, a autenticação de usuários (login) segue usando o scaffolding padrão do Laravel, sem um fluxo de convite ou recuperação de senha customizado — ponto que pode ser objeto de uma evolução futura.

14 Resultados e discussão

A evolução tornou a regra de cobrança auditável e testável de forma isolada, reduzindo o risco de um erro de cálculo chegar a produção sem ser detectado. A separação em Services também tornou os Controllers mais curtos e fáceis de ler, e a introdução de Policies fechou uma brecha de segurança em que qualquer usuário autenticado podia alterar valores financeiros da associação. A nova funcionalidade de Dashboard entrega valor direto ao gestor, que antes precisava consultar a listagem de faturas e somar valores manualmente. Como limitação, a cobertura de testes ainda não inclui os Controllers e as Views (testes de Feature via requisição HTTP completa), ficando restrita à camada de Services e de autorização — uma extensão natural para uma próxima iteração.

15 Considerações finais

Este trabalho evoluiu o Sistema de Controle de Consumo de Água a partir de uma base funcional, aplicando de forma concreta pelo menos cinco conceitos da disciplina:

- Service Layer — regra de negócio extraída dos Controllers para classes de serviço reutilizáveis e testáveis;
- ORM Eloquent e relacionamentos — hasMany, belongsTo e hasOne entre Consumidor, Leitura e Fatura, além de scopes de consulta;
- Migrations — nova coluna e constraint única adicionadas de forma versionada e reversível (métodos up()/down());
- Validação de dados — Form Requests com regras e mensagens centralizadas, desacoplando a validação do Controller;
- Testes automatizados — testes de Feature com PHPUnit cobrindo regra de negócio e autorização, executáveis via php artisan test.

O principal aprendizado foi perceber, na prática, como uma arquitetura em camadas facilita tanto a escrita de testes quanto a evolução seguinte do sistema, além de tornar mais simples justificar tecnicamente cada decisão tomada durante o desenvolvimento.

16 Uso de Inteligência Artificial

Ferramenta utilizada: Claude (Anthropic).

Finalidade do uso: apoio na geração inicial da camada de Services, dos Form Requests, das Policies de autorização e dos testes automatizados de Feature, a partir da descrição do sistema original (README da versão em dupla) e dos requisitos da prova final.

Principais prompts utilizados: pedido para reestruturar a lógica de cálculo de consumo e de fatura em uma camada de Services; pedido para adicionar validação via Form Requests; pedido para adicionar controle de autorização por papel de usuário; pedido para gerar testes automatizados cobrindo as novas regras.

Sugestões aproveitadas: estrutura geral das classes de Service, Form Requests e Policies, e o esqueleto dos testes de Feature.

Modificações realizadas pelo(a) discente:

- **Configuração de ambiente:** Ajuste manual do arquivo .env configurando DB_CONNECTION=sqllite e remoção das variáveis DB_* padrão do MySQL para permitir testes locais rápidos.
- **Persistência de dados:** Execução e validação das migrations e estruturação dos seeders (DatabaseSeeder) para carga de dados dos perfis de gestor e leitorista.
- **Lógica de Negócio e Serviços:** Revisão do serviço de cálculo de faturas (FaturaService) e tratamento de exceções na leitura (LeituraException) para garantir consistência dos dados.
- **Interface do Usuário:** Ajuste e personalização dos arquivos Blade nas views de dashboard, cadastro de consumidor e emissão de faturas.

Problemas identificados nas respostas da ferramenta:

- **Execução no terminal:** Ocorreu erro Could not open input file: artisan porque os comandos sugeridos inicialmente não especificaram que o terminal precisava estar posicionado na raiz da pasta do projeto (consumo-agua).
- **Sincronização com GitHub:** A ferramenta sugeriu dar git push direto, gerando o erro [rejected] main -> main (fetch first) por conta de um README.md remoto, sendo necessário executar um git pull com permissão de históricos não relacionados.
- **Refatoração de Testes:** Ajuste pontual na massa de dados dos testes automatizados (FaturaCalculoTest e AutorizacaoConfiguracaoTaxaTest) para atingir 100% de aprovação na execução do php artisan test.

17 Referências

- LARAVEL. Laravel 12.x Documentation. Disponível em: <https://laravel.com/docs>. Acesso em: 2026.
- PHP. PHP: Hypertext Preprocessor Manual. Disponível em: https://www.php.net/manual/pt_BR/. Acesso em: 2026.
- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: ABNT, 2011.
- INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ. Prova Final — Evolução e Avaliação de uma Aplicação Web em Laravel. Programação Web 1, Campus Boa Viagem, 2026.